

Oswaldo M Cruz Ayala

COMP2052.WEB DEV SERV-
SIDE&MICROSER BKE

Proyecto Final: Back-end

Milagros Donato

Introducción

Este proyecto consiste en una API REST desarrollada utilizando Node.js, Express.js y MongoDB Atlas. La aplicación permite a los usuarios registrarse, iniciar sesión y administrar tareas mediante operaciones CRUD protegidas utilizando autenticación JWT.

El objetivo principal del proyecto fue aplicar conceptos de desarrollo backend, manejo de rutas, conexión con bases de datos y seguridad mediante tokens. Además, se implementó una estructura organizada utilizando modelos, rutas y middlewares para mantener el proyecto profesional y escalable.

Tecnologías Utilizadas

- Node.js
 - Express.js
 - MongoDB Atlas
 - Mongoose
 - JWT (JSON Web Token)
 - bcryptjs
 - dotenv
 - Postman
 - GitHub
-

Descripción del Proyecto

La API REST desarrollada permite realizar operaciones de autenticación y administración de tareas.

Las funcionalidades principales del proyecto incluyen:

- Registro de usuarios
- Inicio de sesión
- Generación de tokens JWT
- Protección de rutas privadas

- Crear tareas
- Obtener tareas
- Actualizar tareas
- Eliminar tareas
- Conexión a MongoDB Atlas

La aplicación fue probada utilizando Postman y el código fue almacenado en un repositorio de GitHub.

Estructura del Proyecto

```
src/  
├── config/  
├── controllers/  
├── middlewares/  
├── models/  
├── routes/  
└── app.js
```

Endpoints Implementados

Metodo	Endpoint	funcion
POST	/api/auth/register	Registrar usuario
POST	/api/auth/login	Iniciar Sesion
POST	/api/task	Crear tarea
GET	/api/task	Obtener Tareas
PUT	/api/task/;id	Actualizar tarea
DELETE	/api/task/;id	Eliminar Tarea

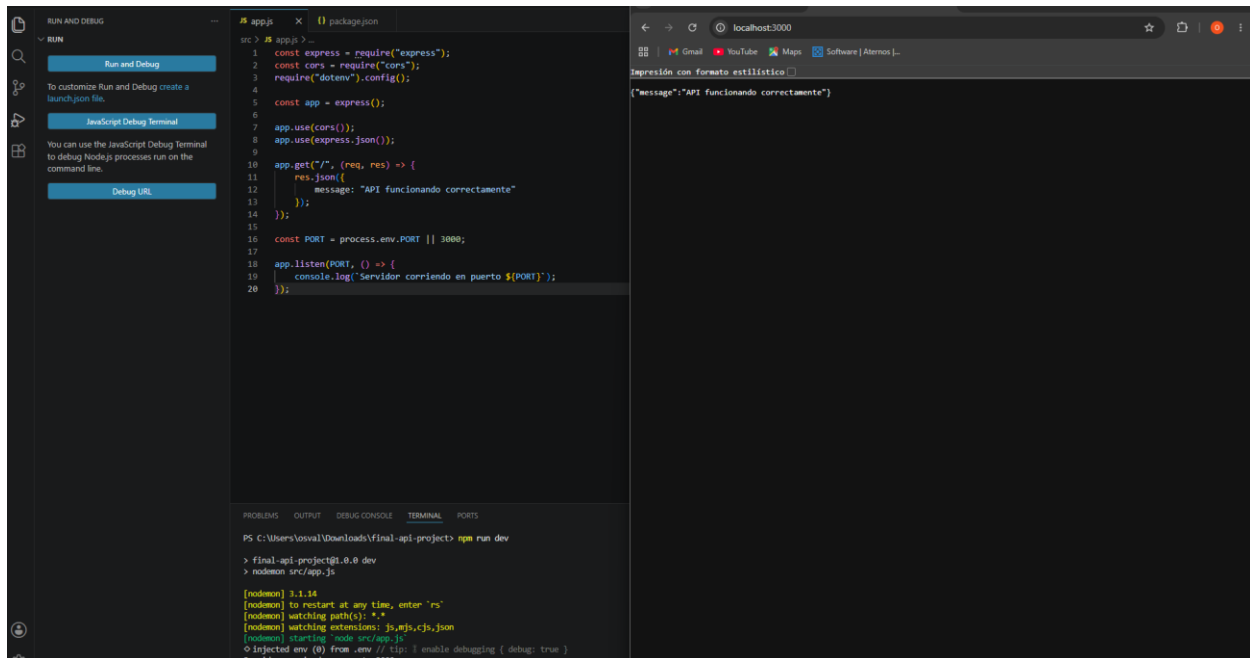
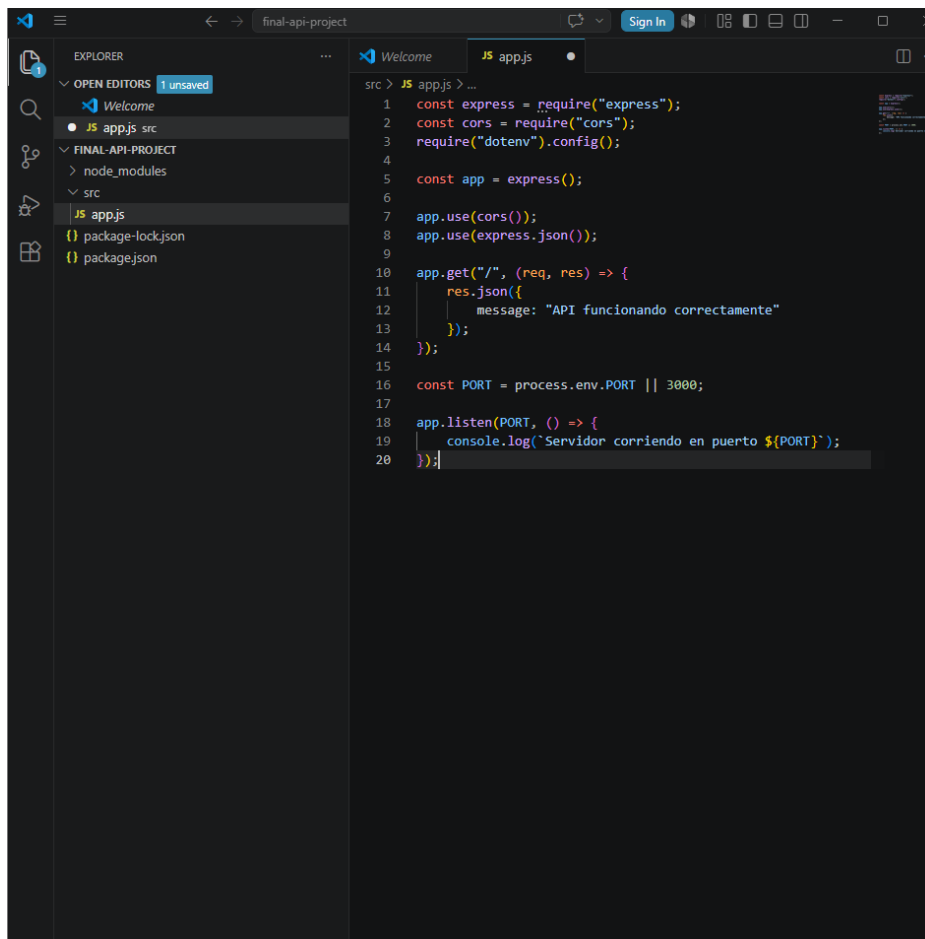
Pruebas Realizadas

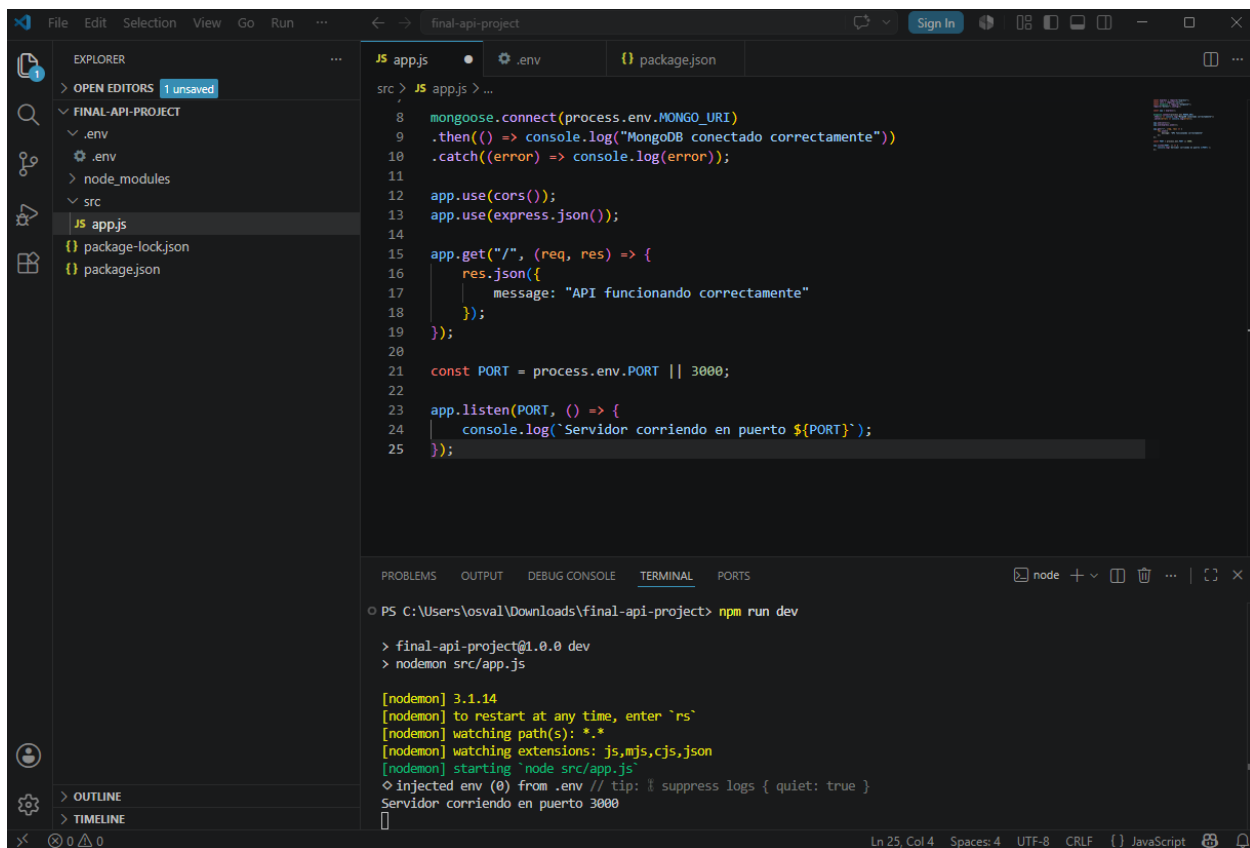
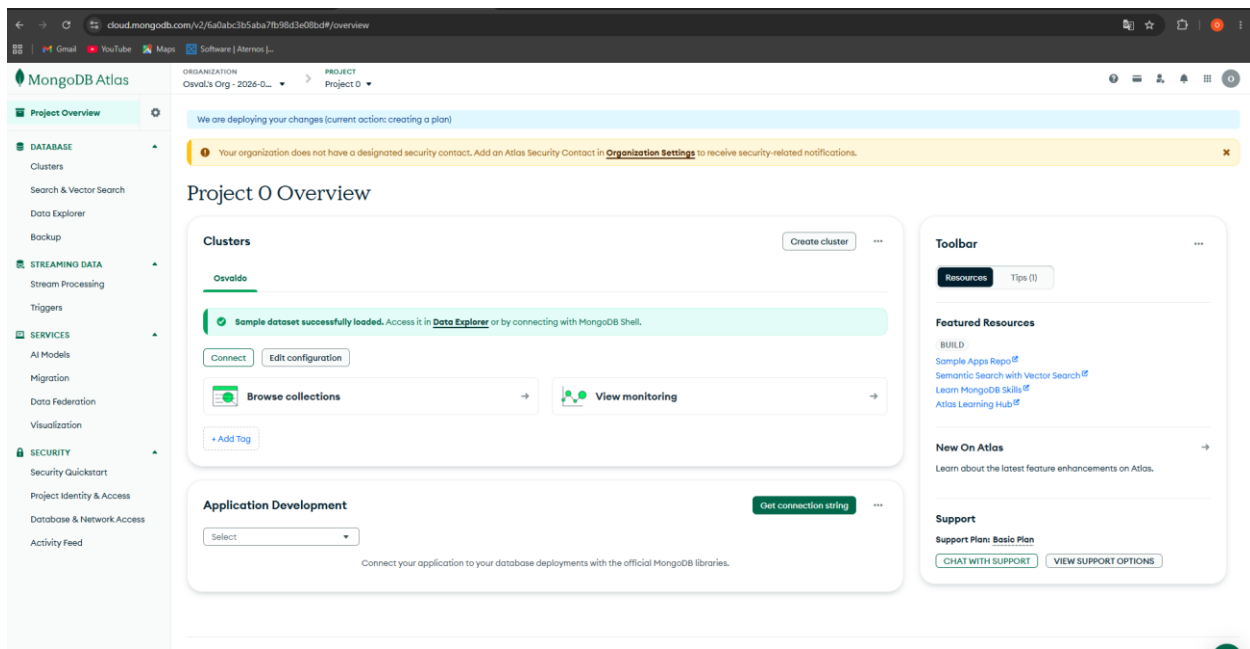
Las pruebas de la API fueron realizadas utilizando Postman.

Se probaron correctamente las siguientes funcionalidades:

- Registro de usuarios
- Login con JWT
- Creación de tareas
- Obtención de tareas
- Actualización de tareas
- Eliminación de tareas

Screenshots





PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\osval\Downloads\final-api-project> npm run dev
at Object..js (node:internal/modules/cjs/loader:1893:10)
at Module.load (node:internal/modules/cjs/loader:1481:32)
at Module._load (node:internal/modules/cjs/loader:1300:12)
at TracingChannel.traceSync (node:diagnostics_channel:328:14)
at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
[nodemon] clean exit - waiting for changes before restart
rs
[nodemon] starting `node src/app.js`
◇ injected env (3) from .env // tip: ◇ encrypted .env [www.dotenvx.com]
Servidor corriendo en puerto 3000
MongoDB conectado correctamente
```

JS app.js • .env • JS User.js • JS Task.js {} package.json ...

```
src > models > JS User.js > ...
1  const mongoose = require("mongoose");
2
3  const userSchema = new mongoose.Schema({
4    email: {
5      type: String,
6      required: true,
7      unique: true
8    },
9
10   password: {
11     type: String,
12     required: true
13   }
14 }, {
15   timestamps: true
16 });
17
18
19 module.exports = mongoose.model("User", userSchema);
```

```
src > models > JS Task.js > ...
1  const mongoose = require("mongoose");
2
3  const taskSchema = new mongoose.Schema({
4
5      title: {
6          type: String,
7          required: true
8      },
9
10     description: {
11         type: String
12     },
13
14     status: {
15         type: String,
16         enum: ["pending", "in_progress", "done"],
17         default: "pending"
18     },
19
20     dueDate: {
21         type: Date
22     },
23
24     user: {
25         type: mongoose.Schema.Types.ObjectId,
26         ref: "User"
27     }
28 }, {
29     timestamps: true
30 });
31
32
33 module.exports = mongoose.model("Task", taskSchema);
34
35
```

```
src > routes > JS authRoutes.js > ...
1  const express = require("express");
2
3  const router = express.Router();
4
5  router.get("/", (req, res) => {
6      res.json({
7          message: "Rutas auth funcionando"
8      });
9  });
10
11 module.exports = router;
```

rc > JS app.js > ...

```
1  const authRoutes = require("../routes/authRoutes");
2  const express = require("express");
3  const cors = require("cors");
4  const mongoose = require("mongoose");
5  require("dotenv").config();
6
7  const app = express();
8
9  mongoose.connect(process.env.MONGO_URI)
10 .then(() => console.log("MongoDB conectado correctamente"))
11 .catch((error) => console.log(error));
12
13 app.use(cors());
14 app.use(express.json());
15 app.use("/api/auth", authRoutes);
16
17 app.get("/", (req, res) => {
18   res.json({
19     message: "API funcionando correctamente"
20   });
21 });
22
23 const PORT = process.env.PORT || 3000;
24
25 app.listen(PORT, () => {
26   console.log(`Servidor corriendo en puerto ${PORT}`);
27 });
28
```

The image shows a VS Code editor with a project named 'FINAL-API-PROJECT'. The file explorer on the left shows the project structure, including 'node_modules', 'src', 'config', 'controllers', 'middlewares', 'models', 'tasks', 'users', 'routes', and 'authRoutes.js'. The main editor displays the 'app.js' file, which is a Node.js application using Express, MongoDB, and CORS. The code is as follows:

```
1 const authRoutes = require("../routes/authRoutes");
2 const express = require("express");
3 const cors = require("cors");
4 const mongoose = require("mongoose");
5 require("dotenv").config();
6
7 const app = express();
8
9 mongoose.connect(process.env.MONGO_URI)
10 .then(() => console.log("MongoDB conectado correctamente"))
11 .catch((error) => console.log(error));
12
13 app.use(cors());
14 app.use(express.json());
15 app.use("/api/auth", authRoutes);
16
17 app.get("/", (req, res) => {
18   res.json({
19     message: "API funcionando correctamente"
20   });
21 });
22
23 const PORT = process.env.PORT || 3000;
24
25 app.listen(PORT, () => {
26   console.log(`Servidor corriendo en puerto ${PORT}`);
27 });
```

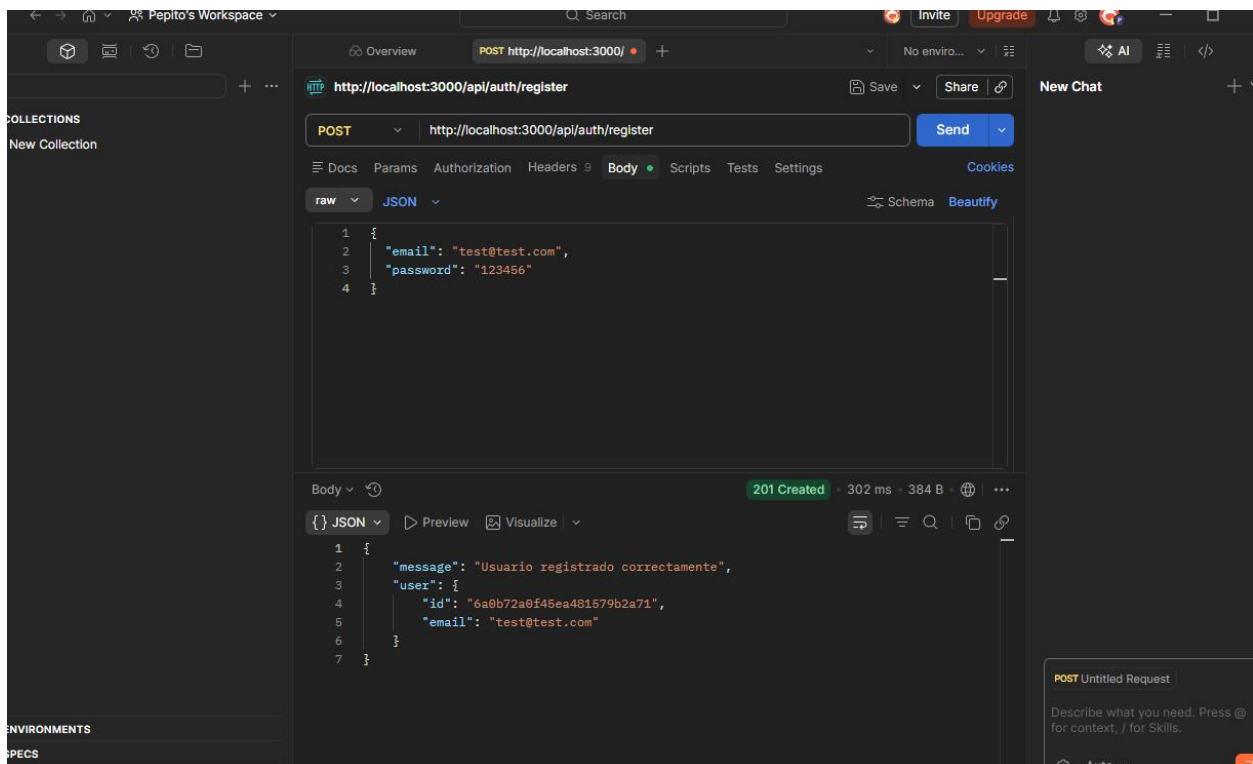
The terminal at the bottom shows the command 'npm run dev' being executed, which starts the application. The output indicates that the application is running on port 3000 and MongoDB is connected. The browser window on the right shows the URL 'localhost:3000/api/auth' and a 404 error message: 'Impresión con formato estilístico' and '({message:"Rutas auth funcionando"})'.

src > routes > JS authRoutes.js > ...

```
6
7 router.post("/register", async (req, res) => {
8   try {
9     const { email, password } = req.body;
10
11     if (!email || !password) {
12       return res.status(400).json({
13         error: true,
14         message: "Email y password son requeridos"
15       });
16     }
17
18     const userExists = await User.findOne({ email });
19
20     if (userExists) {
21       return res.status(400).json({
22         error: true,
23         message: "El usuario ya existe"
24       });
25     }
26
27     const hashedPassword = await bcrypt.hash(password, 10);
28
29     const user = await User.create({
30       email,
31       password: hashedPassword
32     });
33
34     res.status(201).json({
35       message: "Usuario registrado correctamente",
36       user: {
37         id: user._id,
38         email: user.email
39       }
40     });
41
42   } catch (error) {
```

```
> OPEN EDITORS
FINAL-API-PROJECT
  > node_modules
  > src
    > config
    > controllers
    > midlewares
  > models
  JS Task.js
  JS User.js
  > routes
    JS authRoutes.js
  JS app.js
  .env
  package-lock.json
  package.json

src > routes > JS authRoutes.js > ...
7 router.post("/register", async (req, res) => {
41
42   } catch (error) {
43     res.status(500).json({
44       error: true,
45       message: "Error al registrar usuario"
46     });
47   }
48 });
49
50 module.exports = router;
```



OS workspace

Search

Invite Upgrade

Overview

POST http://localhost:3000/

No enviro...

AI

New Chat

HTTP

http://localhost:3000/api/auth/login

Save

Share

Send

POST

http://localhost:3000/api/auth/login

Docs

Params

Authorization

Headers

Body

Scripts

Tests

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {
2   "email": "test@test.com",
3   "password": "123456"
4 }
```

Body

200 OK

148 ms

478 B

...

JSON

Preview

Visualize

```
1 {
2   "message": "Login correcto",
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhMGU3MmEwZjQ1ZWU0ODE1NzliMmE3MSIsIm1hdCI6MTc3OTUzNTI2MSwiZmxwIjoxNzc5MTM4ODYxZjQ7KMzZUj5n6_sU_q5ucuPK6NgWzHxKDq1lJX8xS19EkQ"
4 }
```

POST

Untitled Request

Describe what you need. Press @ for context, / for Skills.

Auto

final-api-project

EXPLORER

1 unsaved

FINAL-API-PROJECT

- node_modules
- src
 - config
 - controllers
 - middlewares
 - JS authMiddleware.js**
 - models
 - JS Task.js
 - JS User.js
 - routes
 - JS authRoutes.js
- app.js
- .env
- package-lock.json
- package.json

src > middlewares > JS authMiddleware.js > ...

```
1 const jwt = require("jsonwebtoken");
2
3 const authMiddleware = (req, res, next) => {
4   const authHeader = req.headers.authorization;
5
6   if (!authHeader) {
7     return res.status(401).json({
8       error: true,
9       message: "No hay token"
10    });
11  }
12
13  const token = authHeader.split(" ")[1];
14
15  try {
16    const decoded = jwt.verify(token, process.env.JWT_SECRET);
17    req.user = decoded;
18    next();
19  } catch (error) {
20    return res.status(401).json({
21      error: true,
22      message: "Token inválido"
23    });
24  }
25 };
26
27 module.exports = authMiddleware;
```

EXPLORER

2 unsaved

FINAL-API-PROJECT

- node_modules
- src
 - config
 - controllers
 - middlewares
 - JS authMiddleware.js
 - models
 - JS Task.js
 - JS User.js
 - routes
 - JS authRoutes.js
 - JS taskRoutes.js**
- app.js
- .env
- package-lock.json
- package.json

src > routes > JS taskRoutes.js > ...

```
1 const express = require("express");
2 const authMiddleware = require("../middlewares/authMiddleware");
3
4 const router = express.Router();
5
6 router.get("/", authMiddleware, (req, res) => {
7   res.json({
8     message: "Rutas de tareas protegidas funcionando",
9     user: req.user
10  });
11 });
12
13 module.exports = router;
```

src > middlewares > JS authMiddleware.js > ...

```
1  const jwt = require("jsonwebtoken");
2
3  const authMiddleware = (req, res, next) => {
4    const authHeader = req.headers.authorization;
5
6    if (!authHeader) {
7      return res.status(401).json({
8        error: true,
9        message: "No hay token"
10      });
11    }
12
13    const token = authHeader.split(" ")[1];
14
15    try {
16      const decoded = jwt.verify(token, process.env.JWT_SECRET);
17      req.user = decoded;
18      next();
19    } catch (error) {
20      return res.status(401).json({
21        error: true,
22        message: "Token inválido"
23      });
24    }
25  };
26
27  module.exports = authMiddleware;
```

Pepito's Workspace

Search

InviteUpgrade

Overview

POST http://localhost:3000/POST http://localhost:3000/

No enviro...

http://localhost:3000/api/tasks

SaveShare

New Chat

POST

http://localhost:3000/api/tasks

Send

DocsParamsAuthorizationHeaders10BodyScriptsTestsSettingsCookies

rawJSONSchemaBeautify

```
1 {
2   "title": "Proyecto Final",
3   "description": "Terminar API REST",
4   "status": "pending"
5 }
```

Body201 Created · 90 ms · 505 B

JSONPreviewVisualize

```
1 {
2   "title": "Proyecto Final",
3   "description": "Terminar API REST",
4   "status": "pending",
5   "user": "6a9b72a8f45ea481579b2a71",
6   "_id": "6a9b7c6b5cd78544783de792",
7   "createdAt": "2026-05-18T20:54:03.646Z",
8   "updatedAt": "2026-05-18T20:54:03.646Z",
9   "__v": 0
10 }
```

ENVIRONMENTS

SPECS

FLOWS

Connect GitConsoleTerminal

GlobalsVaultTools

POST http://localhost:3...

Describe what you need. Press @ for context, / for Skills.

Auto

PUT

http://localhost:3000/api/tasks/6a0b7c6b5cd70544703de792

Send

Docs

Params

Authorization

Headers 10

Body

Scripts

Tests

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {
2   "status": "done"
3 }
```

Body

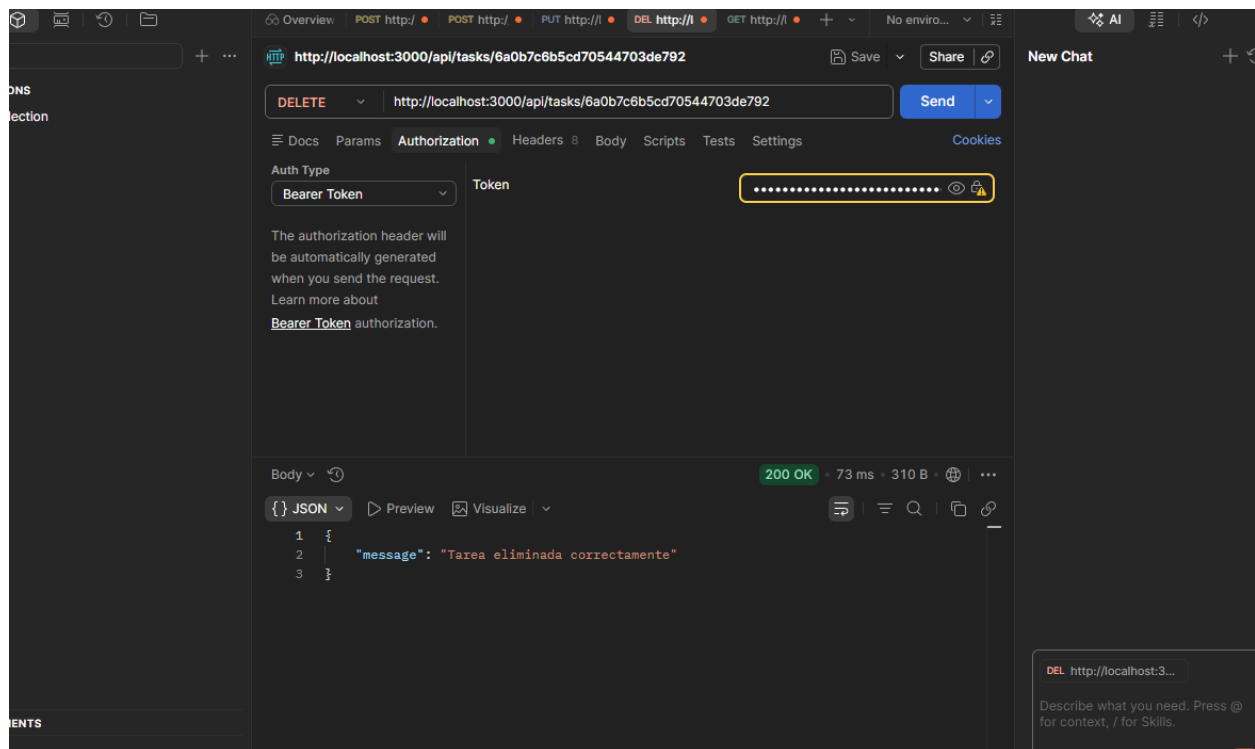
200 OK • 170 ms • 497 B •

JSON

Preview

Visualize

```
1 {
2   "_id": "6a0b7c6b5cd70544703de792",
3   "title": "Proyecto Final",
4   "description": "Terminar API REST",
5   "status": "done",
6   "user": "6a0b72a0f45ea481579b2a71",
7   "createdAt": "2026-05-18T20:54:03.646Z",
8   "updatedAt": "2026-05-18T20:58:45.825Z",
9   "__v": 0
10 }
```



POST

http://localhost:3000/api/tasks

Send

Docs

Params

Authorization

Headers 10

Body

Scripts

Tests

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

201 Created • 90 ms • 505 B | Save Response

JSON

Preview

Visualize

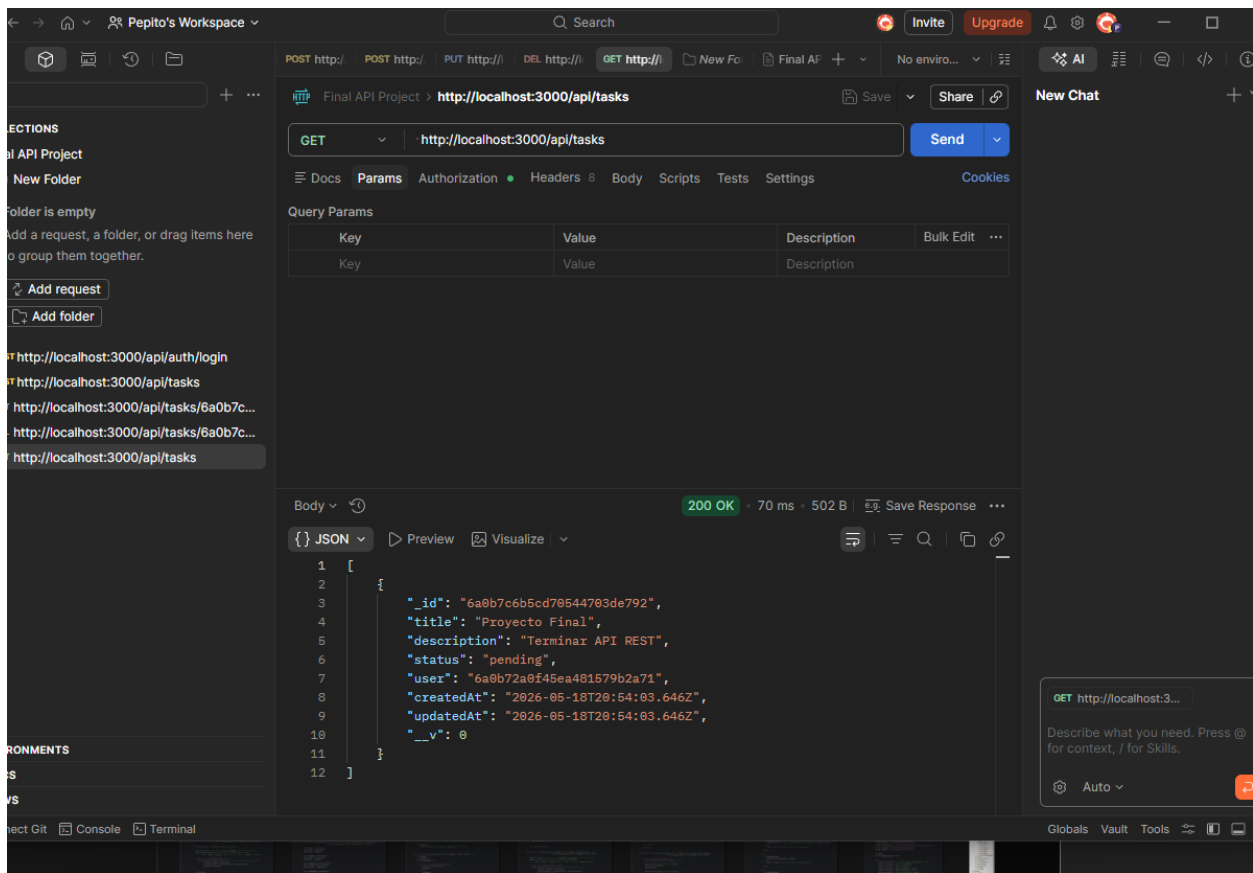
```
1 {
2   "title": "Proyecto Final",
3   "description": "Terminar API REST",
4   "status": "pending",
5   "user": "6a0b72a0f45ea481579b2a71",
6   "_id": "6a0b7c6b5cd70544703de792",
7   "createdAt": "2026-05-18T20:54:03.646Z",
8   "updatedAt": "2026-05-18T20:54:03.646Z",
9   "__v": 0
10 }
```

POST http://

Describe v

for contex

Auto



Conclusión

Durante el desarrollo de este proyecto se logró implementar una API REST funcional utilizando tecnologías modernas de backend. Además, se aplicaron conceptos importantes como autenticación JWT, manejo de bases de datos NoSQL y operaciones CRUD.

El proyecto permitió fortalecer conocimientos en desarrollo backend, seguridad utilizando JWT y manejo de bases de datos con MongoDB Atlas. También se desarrollaron habilidades utilizando herramientas profesionales como Postman y GitHub.

Finalmente, el proyecto cumple con los requisitos establecidos y demuestra el funcionamiento completo de una API REST moderna y funcional.